



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного бюджетного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК «Информатика и управление»

КАФЕДРА ИУК5 «Системы обработки информации»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ

НА ТЕМУ:

*Разработка многопользовательской браузерной игры "Рогоз" с
использованием веб-сокетов.*

Студент группы ИУК5-62Б

(подпись, дата)

Коммер К. А.

(Фамилия И.О.)

Руководитель курсового проекта

(подпись, дата)

Фролов П. А.

(Фамилия И.О.)

Калуга, 2024

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного бюджетного образовательного
учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой ИУК5
(Вершинин Е.В.)
« ____ » _____ 2024 г.

ЗАДАНИЕ

на выполнение курсового проекта

по дисциплине Сетевые технологии в автоматизированных системах
обработки информации и управления

Студент группы ИУК5-62Б Коммер Кристиан Айгарович
(фамилия, имя, отчество)

Тема курсового проекта Разработка многопользовательской браузерной игры
"Рогоз" с использованием веб-сокетов.

Направленность КП учебный

Источник тематики кафедра ИУК5

Задание

Разработать клиент-серверное приложение с использованием веб-сокетов.
Оформить расчетно-пояснительную записку и .

Оформление курсового проекта

Расчетно-пояснительная записка на _____ листах формата А4.

Перечень графического материала КП (плакаты, схемы, чертежи и т.п.):

- Скриншоты интерфейса – 1 лист формата А3;
- Sequence диаграмма главного процесса игры – 1 лист формата А3;
- Sequence диаграммы таблицы лидеров и чата – 1 лист формата А3

Дата выдачи задания « ____ » _____ 2024 г.

Руководитель

(подпись, дата)

Фролов П. А.

(Фамилия И.О.)

Студент

(подпись, дата)

Коммер К. А.

(Фамилия И.О.)

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного бюджетного образовательного
учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

КАЛЕНДАРНЫЙ ПЛАН
на выполнение курсового проекта

по дисциплине **Сетевые технологии в автоматизированных системах
обработки информации и управления**

Студент группы **ИУК5-62Б Коммер Кристиан Айгарович**
(фамилия, имя, отчество)

Тема курсового проекта **Разработка многопользовательской браузерной игры
"Рогоз" с использованием веб-сокетов.**

№	Наименование этапов	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Руководитель	Куратор
1	Задание на выполнение	1-я нед.			
2	Выполнение основных расчетов и эскизов графической части	10-я нед.			
3	Выполнение и окончательное оформление графической части и расчетно-пояснительной записки	14-я нед.			
4	Защита	17-я нед.			

Студент _____ Руководитель _____
(подпись, дата) (подпись, дата)

СОДЕРЖАНИЕ

Введение.....	5
1. Техническое задание.....	6
1.1 Общие сведения.....	6
1.2 Назначения и цели создания системы.....	6
1.3 Требования к системе.....	7
1.4 Состав и содержание работ по созданию (развитию) системы.....	8
1.5 Порядок контроля и приемки системы.....	9
1.6 Требования к документированию.....	9
2. Научно-исследовательская часть.....	10
2.1 Постановка задачи проектирования.....	10
2.2 Описание предметной области.....	10
2.3 Анализ аналогов и прототипов.....	11
2.4 Перечень задач подлежащих решению в процессе разработки.....	14
2.5 Обоснование выбора инструментов и платформы для разработки.....	14
2.6 Требования к приложению.....	15
3. Проектно-конструкторская часть.....	16
3.1 Реализация готового приложения.....	16
3.2 Разработка и реализация структуры приложения.....	16
3.3 Разработка алгоритмов связи приложения.....	18
3.4 Разработка алгоритмов обработки информации.....	21
3.5 Разработка интерфейса взаимодействия пользователя с системой.....	21
4. Проектно-технологическая часть.....	26
4.1 Тестирование и отладка готового модуля.....	26
4.2 Руководство администратора.....	27
4.3 Руководство пользователя.....	30
Заключение.....	32
Список литературы.....	33
Приложение.....	34

Введение

Для получения удовольствия от игры немалую роль играют другие люди. Социализация - неотъемлемая часть нашей жизни. В игры можно играть с друзьями, одноклассниками, родными или даже незнакомцами, однако на большом расстоянии в настольные игры поиграть не получится, поэтому существуют онлайн многопользовательские игры, которые связывают людей, находящихся на разных уголках света.

Суть игры: получение наибольшего количества очков среди игроков. Игрок находится на болоте среди камышей - рогозов, он может кидать рогоз во врага и наносить ему урон, после получения достаточного количества урона игрок нокаутируется и игроку, нанёсшему последний удар засчитывается один балл. После нокаутации игрок магической силой переносится в другое место и бой продолжается с теми же правилами.

В игре присутствуют различные кратковременные улучшатели: восстановление очков здоровья, увеличение скорости стрельбы, выпуск нескольких камышей одновременно и ускорение передвижения. Все эти улучшатели прорастают в болоте среди обычных камышей и рогозов, но отличаются по внешнему виду, игрок может использовать окружение и эти кратковременные улучшения для получения тактического преимущества над врагами.

1. Техническое задание

1.1 Общие сведения

1.1.1 Полное наименование системы:

Многопользовательская браузерная 2D игра - в жанре экшен, основанная на HTML5.

1.1.2 Наименование организации-заказчика, разработчика системы:

Заказчик: КФ МГТУ им. Н. Э. Баумана

Разработчик: Студент КФ МГТУ им. Н. Э. Баумана группы ИУК5-62Б
Коммер К.А.

1.1.3 Плановые сроки начала и окончания работы по созданию системы:

Плановый срок начала работ 16 марта 2024 года.

Плановый срок окончания работ 31 мая 2024 года.

1.2 Назначения и цели создания системы

1.2.1 Назначение системы:

Система предназначена для связи пользователей друг с другом через сервер для совместной игры.

1.2.2 Цели создания системы:

Система создается для реализации соединения между сервером и клиентом посредством сетевых взаимодействий.

1.3 Требования к системе

1.3.1 Требования к системе в целом

1.3.1.1 Требования к структуре и функционированию системы:

Система разделена на 2 подструктуры:

- Клиентское веб-приложение
- Серверное приложение

1.3.1.2 Требования по эргономике и технической эстетике:

Интерфейс системы не должен вводить пользователя в заблуждение, содержать элементы, назначение которых не очевидно или элементы, функционал которых отсутствует или не соответствует пояснению в интерфейсе.

Интерфейс должен быть понятен и не перегружен графическими элементами.

1.3.2 Требования к функциям (задачам), выполняемым системой:

- Подключение клиента к серверу с созданием сеанса
- Отправка и получение сообщений с сервера используя протокол TCP
- Моделирование геометрии на сервере

1.3.3 Требования к видам обеспечения

1.3.3.1 Требования к лингвистическому обеспечению системы:

Средством описания предметной области, также, как и средством взаимодействия пользователя с системой является русский язык.

1.3.3.2 Требование к входным и выходным данным:

Входными данными от игрока являются: временный псевдоним персонажа, input клавиатуры.

Выходными данными являются: отображение пространства сервера.

1.3.3.3 Требования к программному обеспечению системы:

В качестве языка программирования клиентской и серверной части игры используется JavaScript.

1.4 Состав и содержание работ по созданию (развитию) системы:

Таблица 1.1 - План график

№	Состав работ	Промежуток времени
1	Утверждение темы, задания на разработку, технического задания. Оформление ТЗ. Описание предметной области и требований к системе. Прототип или скетчи интерфейса.	6 - 7-я недели
2	Оформление введения и исследовательской части. Демонстрация работы макета системы.	8 - 9-я недели
3	Оформление проектно-конструкторской части. Демонстрация работающего приложения. Тестирование и отладка приложения. Разработка эксплуатационной документации.	10 - 11-я недели
4	Завершающее оформление документации согласно требованиям ГОСТ. Все ошибки и проблемы устранены. Подготовка доклада. Защита курсовой работы.	12 - 13-я недели

1.5 Порядок контроля и приемки системы:

Система разрабатывается с применением тестирования работоспособности функционала. При добавлении новых функций предыдущие тесты должны сохранять работоспособность.

Приемка работы осуществляется комиссией КФ МГТУ им. Н. Э. Баумана кафедры ИУК5.

1.6 Требования к документированию:

Требуется предоставить:

- a) Техническое задание в соответствии с ГОСТ 34.602-89
- b) Расчетно-пояснительную записку, включающую исследовательскую часть, проектно-конструкторскую технологическую часть, включающую в себя руководство пользователя и руководство программиста (администратора). Расчетно-пояснительная записка выполняется с учетом требований, предусмотренных ГОСТ 7.32-2001 и 2.105-95.

2. Научно-исследовательская часть

2.1 Постановка задачи проектирования

Задачами проектирования являются:

1. Спроектировать архитектуру приложения.
2. Разработать клиентскую часть приложения.
3. Разработать серверную часть приложения.
4. Оформить документацию.

2.2 Описание предметной области

Система представляет собой многопользовательскую игру “Рогоз”, которая включает в себя: поле боя, игроков, сущности(рогоз, улучшатели), летящие рогозы(которые выпустили игроки), таблицу лидеров и общий чат. После ввода уникального никнейма, игрока забрасывает на игровое поле - болото, у игрока неограниченное количество рогозов, которое он может кинуть, чтобы нокаутировать своих соперников, однако если рогоз попадёт с рогоз, то они самоуничтожатся. Поскольку на игровом поле очень много рогоза - задача сильно усложняется, так как приходится в буквальном смысле прорубать себе путь сквозь заросли. Игроки могут съесть кратковременные улучшатели, которые растут рядом с рогозом, тем самым усилив скорость стрельбы, передвижения или восполнить своё здоровье. После нокаутирования соперника, в таблице лидеров у игрока прибавляется очко и он поднимается выше по рейтингу. Пользователи могут общаться в общем чате, сообщение, которое написал один - видно всем, так же сервер оповещает пользователей в чате при подключении и отключении игроков. Для безопасности на клиенте не производится никаких вычислений, все данные с клавиатуры и мыши отправляются на сервер 60 раз в секунду, а сервер отправляет клиенту только расположение сущностей в игре.

2.3 Анализ аналогов и прототипов

1. agar.io:

Agar.io — бесплатная двумерная массово-многопользовательская онлайн-игра 2015 года, созданная бразильским разработчиком Матеус Валадарес. В ней клетки бактерий передвигаются в чашке Петри, поглощая гранулы пищи, а также едят друг друга. Является браузерной игрой, также имеется мобильная версия для Android и iOS. Игра была названа в честь агар-агара (англ. agar) — питательной среды для разведения бактерий. Agar.io получила положительные отзывы критиков, которые особенно хвалили простоту и механику проекта. К негативным моментам игры отнесли повторяющийся игровой процесс.



Рисунок 2.1 - Игровой процесс Agar.io

Межсетевое взаимодействие: Использует WebSockets для обмена данными в реальном времени. Это позволяет игрокам видеть действия друг друга и реагировать на них.

Скорость и надежность: Требуется стабильного соединения для корректной работы, так как игроки должны быстро реагировать на изменения в игровом мире.

2. slither.io:

Slither.io — многопользовательская компьютерная игра, разработанная Стивом Хаузом и выпущенная для iOS, Android и веб-браузеров в марте 2016 года. По своей концепции Slither.io похожа на классическую «Змейку» и другую популярную браузерную игру Agar.io. Игроки управляют «змеями» — длинными существами, которые передвигаются по обширному игровому полю и поедают разбросанные по нему разноцветные гранулы, становясь длиннее. Цель игрока — сделать свою змею как можно длиннее, избегая столкновений со змеями других игроков. Игра возымела огромную популярность после многочисленных летсплеев на YouTube. Согласно рейтингу Alexa за июль 2016 года, браузерная версия Slither.io попала в топ самых посещаемых сайтов, а версия для iOS возглавила чарт самых загружаемых игр в App Store. Игра получила в основном положительные отзывы от критиков — основными её плюсами называли дизайн и кастомизацию змеи, но в качестве недостатков отмечали низкую реиграбельность и высокую цену за удаление рекламы.

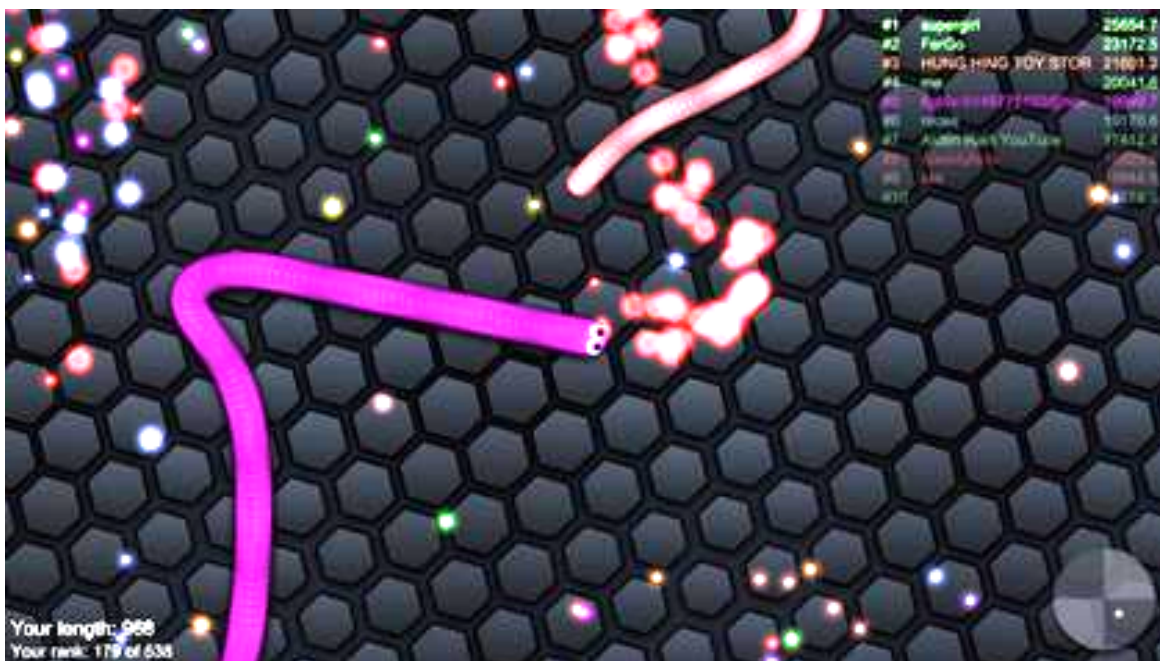


Рисунок 2.2 - Игровой процесс Slither.io

Межсетевое взаимодействие: Также использует WebSockets для синхронизации игрового процесса между клиентом и сервером.

Скорость и надежность: Как и в agar.io, надежное и быстрое соединение необходимо для предотвращения задержек и неправильного отображения действий других игроков.

3. diep.io:

Diep.io — многопользовательская браузерная игра, созданная бразильским разработчиком Матеусом Валадаресом в 2016 году. Она также послужила источником вдохновения для Agras.io. Miniclip впервые опубликовал мобильную версию. В Diep.io игроки управляют танками на двухмерной арене. Они зарабатывают очки опыта и улучшения, уничтожая фигуры и другие танки. Валадарес создал аналогичный Agar.io в 2015 году. Его вирусная популярность вдохновила на создание жанра подобных браузерных игр с простым игровым процессом и графикой под названием « игры .io », в который входит Diep.io. Diep.io хвалили за систему обновлений, которая придавала ей большую глубину и ценность воспроизведения , чем другие игры «.io». В 2021 году ее приобрела компания Addicting Games . Google Play сообщает о более чем 10 миллионах мобильных загрузок по состоянию на 2023 год.



Рисунок 2.3 - Игровой процесс Diep.io

Межсетевое взаимодействие: Использует подобный подход с WebSockets для обмена данными о положении танков, их действиях и состоянии игры.

Скорость и надежность: Высокая скорость обновления данных и надежность соединения критичны для игры, где точность и скорость реакции имеют решающее значение. В `diar.io` больше данных для обмена из-за большего количества объектов на карте и более сложных взаимодействий между ними по сравнению с другими играми.

С рассматриваемым в этой работе приложением все эти игры объединяет использование веб-сокетов и обработка на сервере, все они сыскали популярность благодаря своей простоте, доступности и уникальности игрового геймплея. `Rogoz` также имеет свой уникальный и простой геймплей.

2.4 Перечень задач подлежащих решению в процессе разработки

1. Создать тестовый прототип клиент-серверной архитектуры.
2. Разработать клиентскую и серверную часть приложения.
3. Настроить связь между серверной и клиентской частями приложения.
4. Разработать дружелюбный интерфейс.
5. Провести тесты системы.

2.5 Обоснование выбора инструментов и платформы для разработки.

Для разработки был выбран фреймворк `Node.js` на JavaScript поскольку в сравнении с традиционными серверными технологиями, такими как Java или .NET, `Node.js` предлагает более легковесное и быстрое решение, особенно подходящее для современных веб-приложений и сервисов. Также у `Node.js` большое комьюнити, что позволяет быстро решить все возникающие проблемы.

Клиент ведёт работу с приложением с помощью браузера и HTML5. С помощью элемента HTML5 Canvas происходит отрисовка. Сравнение Canvas с аналогами: SVG: SVG также используется для создания векторной графики в браузере, но он менее производителен, чем Canvas, и не подходит для создания сложных анимаций.

WebGL: WebGL - это еще один вариант для создания 3D-игр в браузере. Он более мощный, чем Canvas, но сложнее в использовании и не так хорошо поддерживается всеми браузерами.

Библиотеки игровых движков: Существуют различные игровые движки, такие как Phaser и Babylon.js, которые можно использовать для создания игр на HTML5. Эти библиотеки упрощают процесс разработки игр, но они менее гибкие, чем Canvas.

2.6 Требования к приложению

1. Прозрачность.
2. Надёжность.
3. Лёгкость для сервера и клиента.

3. Проектно-конструкторская часть

3.1 Реализация готового приложения.

Для решения задач и создания приложения, обладающего всеми вышеперечисленными функциями, необходимо понять, как будут применяться HTML5 и Node.js

3.2 Разработка и реализация структуры приложения

Для реализации структуры приложения была разработана sequence uml диаграммы для игры, чата и таблицы лидеров. Данные диаграммы приведены ниже:

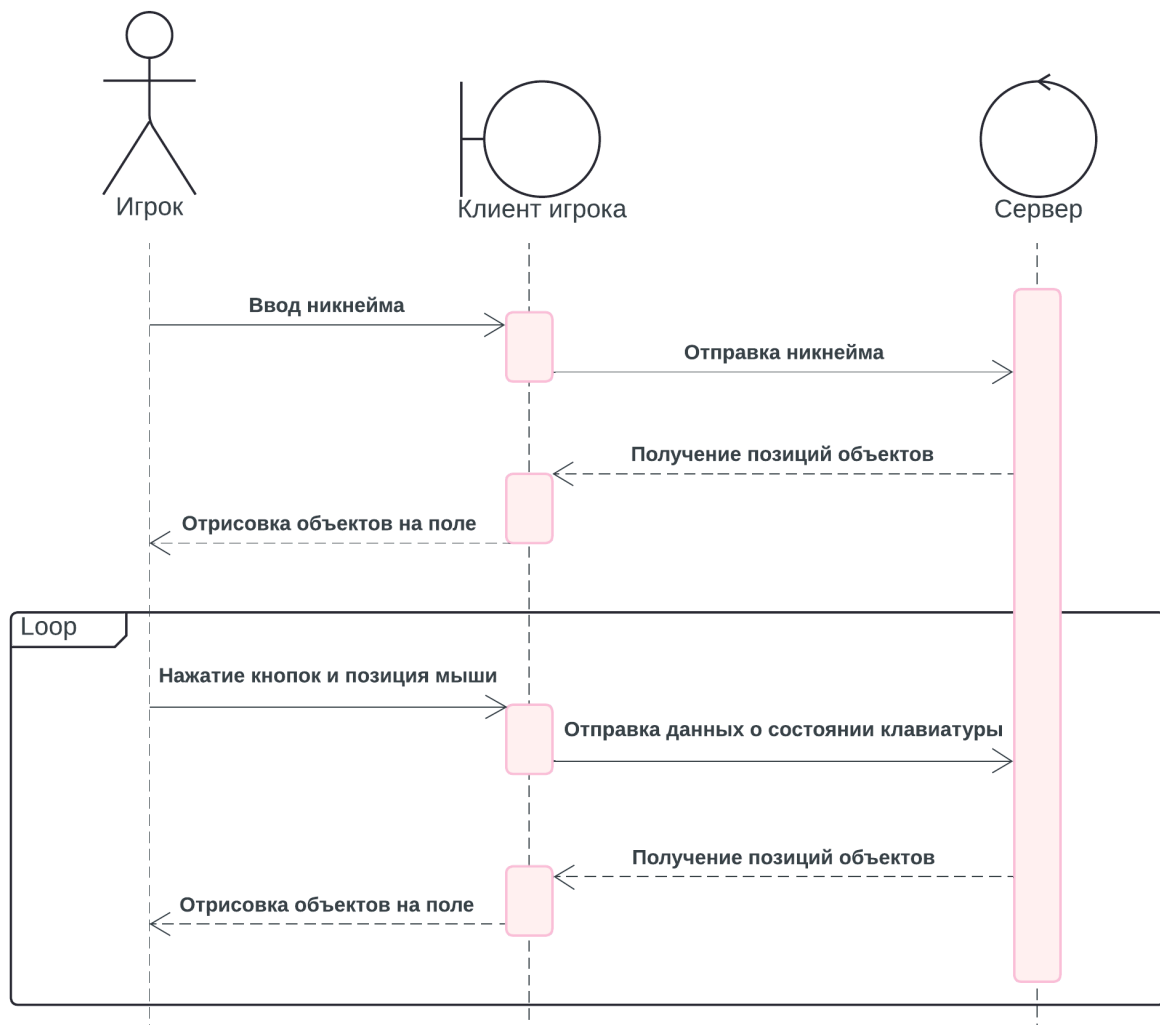


Схема 3.1 – Sequence диаграмма главного процесса игры

На Схеме 3.1 показан основной процесс игры: сначала пользователь вводит никнейм и сервер проверяет правильность никнейма, после чего игрок попадает в игровое пространство, сервер 60 раз в секунду присылает координаты объектов на поле, а клиент получает их и рисует объекты в HTML Canvas окне для игрока. Клиент в свою очередь отправляет состояние клавиш wasd, положение мыши и состояние левой кнопки мыши, сервер получает эту информацию и моделирует изменения.

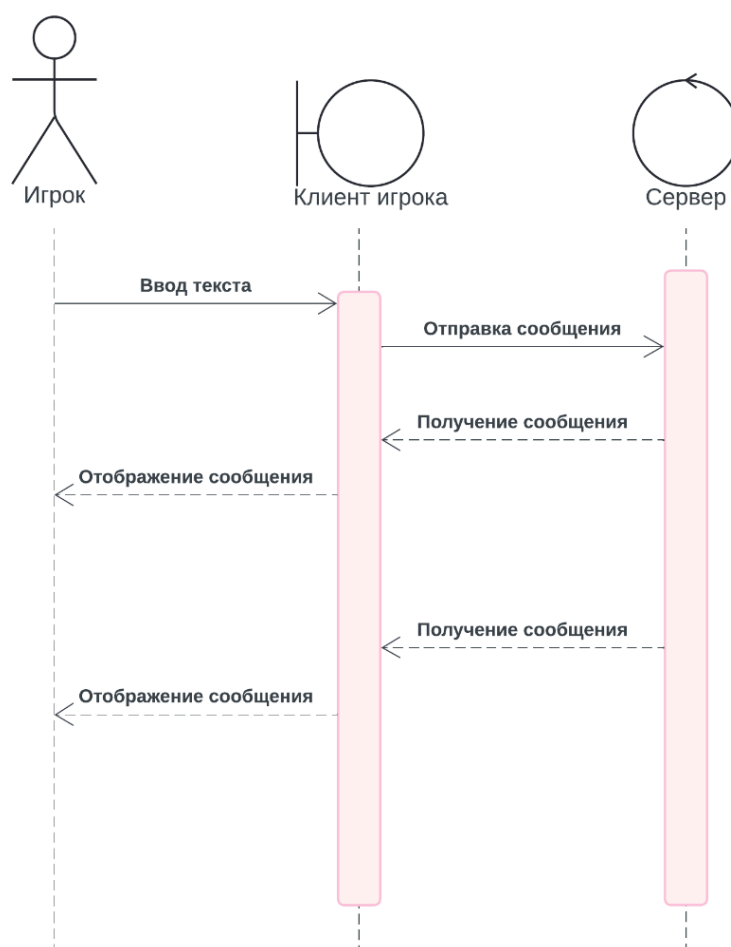


Схема 3.2 – Sequence диаграмма чата

На схеме 3.2 показано межсетевое взаимодействие клиента и сервера при работе с чатом: при вводе текста пользователем, клиент формирует сообщение и отправляет его серверу, сервер получает и рассылает это сообщение всем подключенным к игре пользователям.

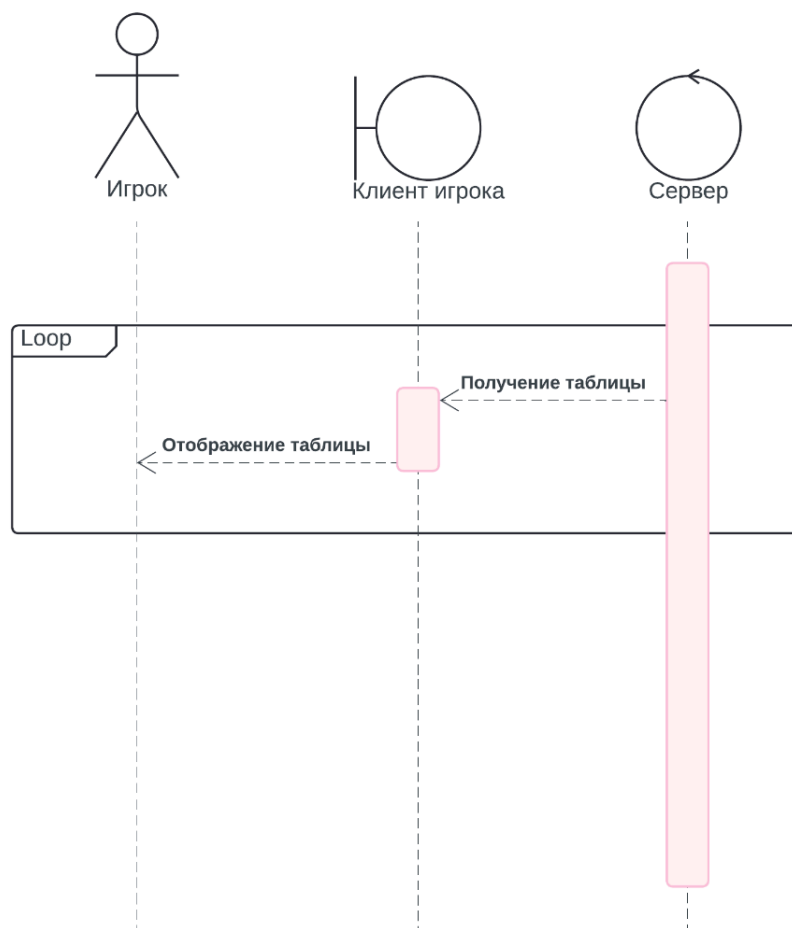


Схема 3.3 – Sequence диаграмма таблицы лидеров

На схеме 3.3 показано межсетевое взаимодействие клиента и сервера при работе с таблицей лидеров: поскольку всё моделирование происходит на сервере, то клиент не имеет доступ к таблице лидеров, он только получает с сервера асинхронное сообщение об изменении таблицы и отрисовывает изменение для пользователя.

3.3 Разработка алгоритмов связи приложения

Создаётся основной файл сервера, в котором выполняется приём данных с клиента.

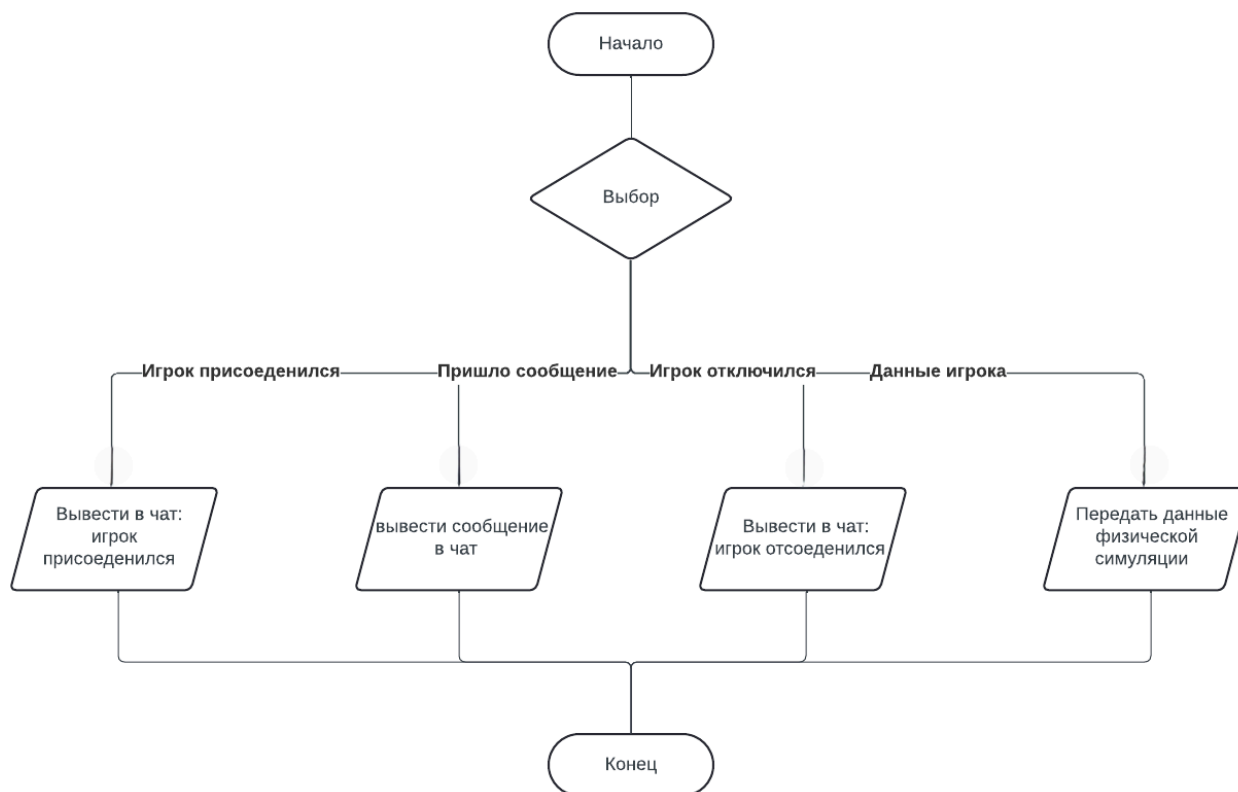


Схема 3.1 – Приём пакета данных сервером

“Смотри приложение №1” При присоединении нового игрока или выхода его сервер оповещает всех в чате, если пользователь сам захочет отправить сообщение, то ему в этом поможет функция `CHAT_CLIENT_SERVER`. Здесь `socket` - это веб-сокет, благодаря которому осуществляется передача и приём данных извне. `game.updatePlayerOnInput(socket.id, data)` Отвечает за обновление данных о нажатых игроком кнопках. Со стороны клиента функция присоединения на сервер и проверки псевдонима выглядит следующим образом: “Смотри приложение №2”

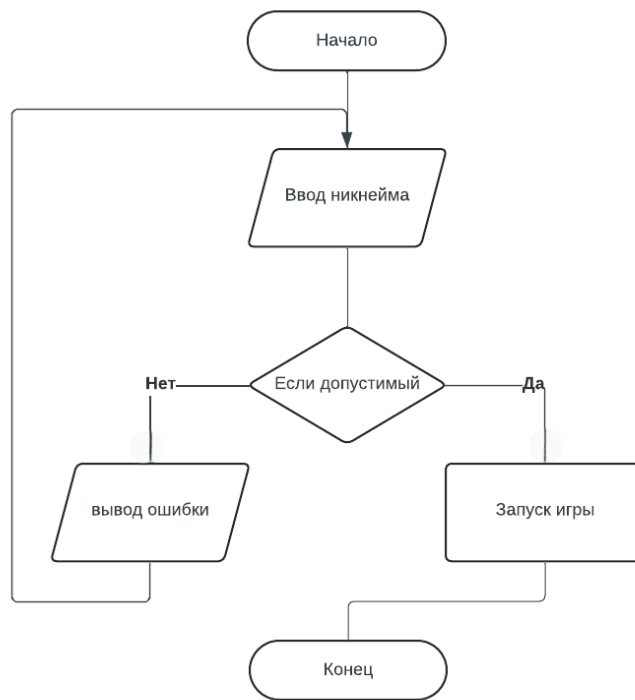


Схема 3.2 – Присоединение на сервер

Данная функция на стороне клиента инициализирует основные функции приложения, такие как Canvas, для отображения поля боя. Далее из HTML контейнера получает имя пользователя и в случае успеха, отправляет данные на сервер. Сервер в свою очередь отправляет данные клиенту с помощью функции:

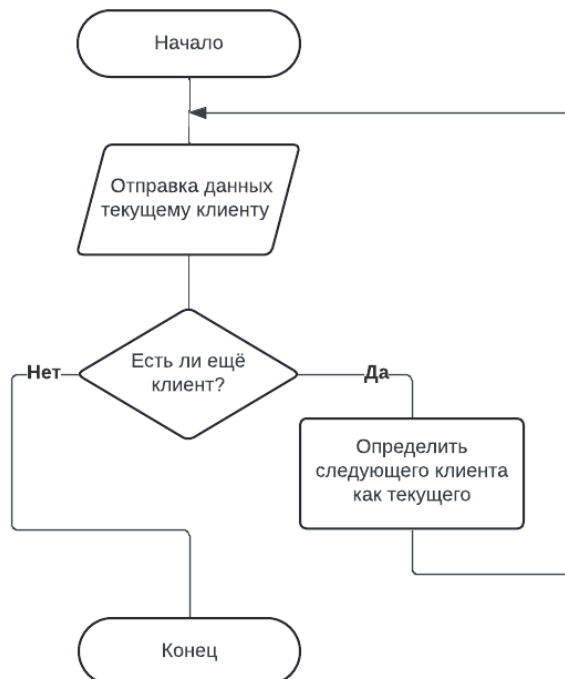


Схема 3.3 – Отправка данных клиентам

“Смотри приложение №3”

Функция `sendState()` отправляет такие данные как: `currentPlayer`(Координаты текущего игрока), `players`(координаты остальных игроков), `projectiles`(координаты летящих рогозов), `powerups`(координаты рогозов и камышей, которые растут на болоте)

3.4 Разработка алгоритмов обработки информации

Обработка всех сущностей на сервере осуществляется следующими функциями: “Смотри приложение №4”

В данном фрагменте кода отрабатывается логика сервера с столкновениями сущностей, как игроков, так и рогозов. `e1` отвечает за первую сущность столкновения, `e2` - за вторую.

Обработка нажатия клавиш игроком на сервере: “Смотри приложение №5”

В данном фрагменте кода функция `updateOnInput(data)` получает данные о нажатых кнопках с клиента и в зависимости от этого двигает модель игрока в игровом пространстве.

Для того, чтобы игрок мог кидать рогозы используются следующие функции:

“Смотри приложение №6”

Здесь первая функция `createFromPlayer` создает летящий рогоз по направлению руки игрока, а вторая функция наблюдает за состоянием и пройденной дистанцией рогоза, чтобы вовремя его уничтожить.

На клиенте отрисовка полученной информации выглядит следующим образом: “Смотри приложение №7”

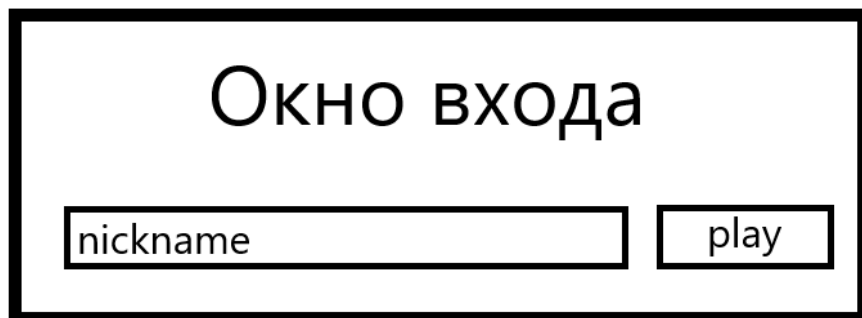
Данная функция осуществляет контроль отрисовки всех объектов. Функции получают пространственные координаты объектов и отрисовывают их на игровом поле. Всё отображение осуществляется в элементе `Canvas`.

3.5 Разработка интерфейса взаимодействия пользователя с системой

Взаимодействие пользователя с системой производится с помощью использования внутриигрового пользовательского интерфейса.

Элементами интерфейса будут блоки, рамки, кнопки.

В начале игры пользователя будет встречать окно, предлагающее ввести никнейм:



The image shows a simple rectangular window with a thick black border. Inside, the text "Окно входа" (Login Window) is centered at the top. Below it, there is a text input field containing the placeholder text "nickname" and a button labeled "play" to its right.

Рисунок 3.1 – Прототип окна ввода никнейма

В поле `nickname` игрок вводит свой псевдоним, кнопка `play` запускает цикл проверки псевдонима, и в случае успеха инициализирует вход в игру.

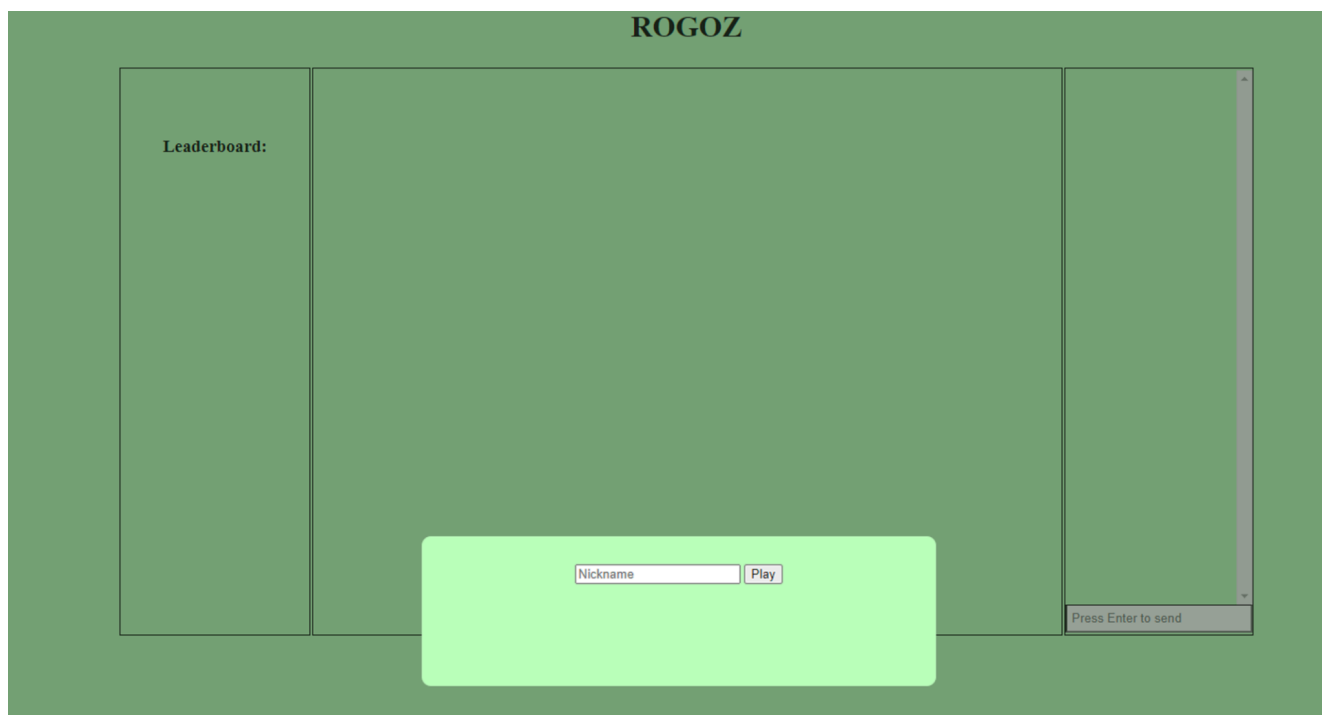


Рисунок 3.2 – Окно ввода никнейма

При входе пользователя автоматически перебрасывает на игровое поле, подключается чат и таблица лидеров:

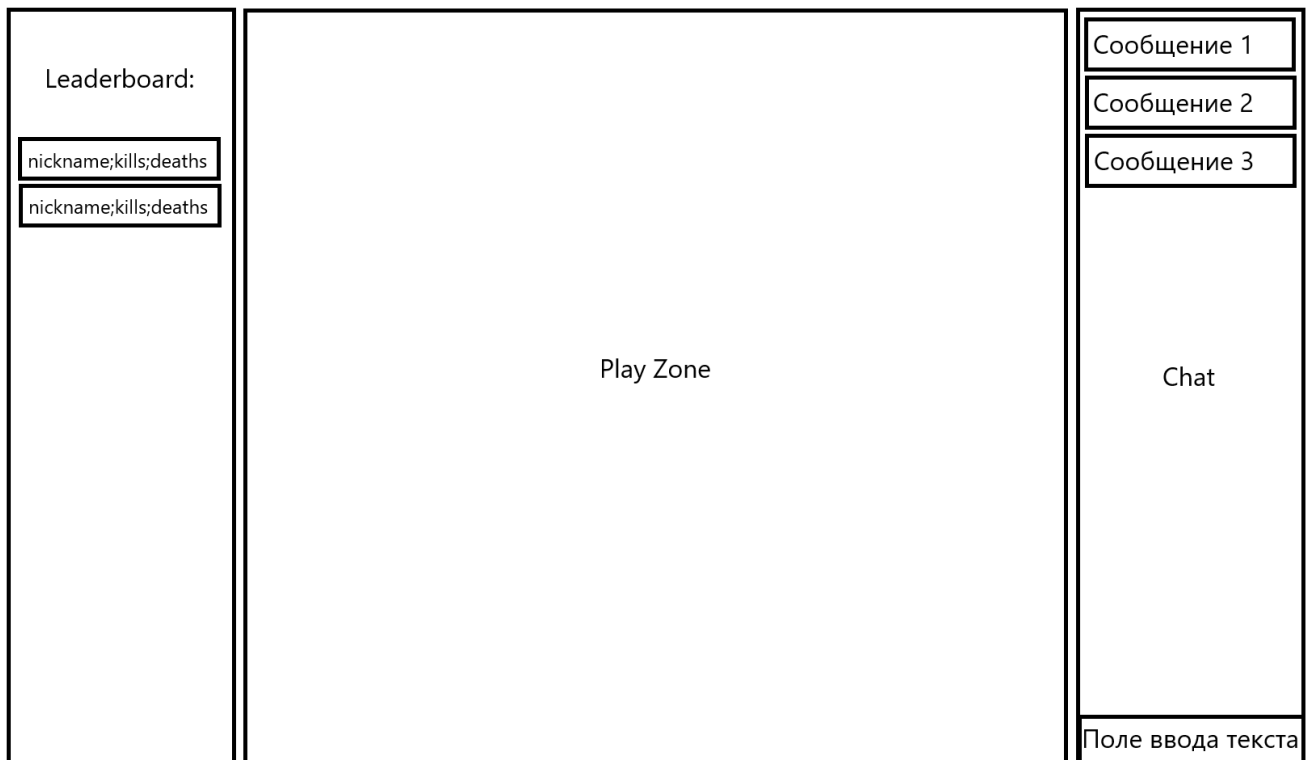


Рисунок 3.3 – Прототип главного окна

Leaderboard - таблица лидеров, здесь списком выводятся все игроки на сервере, их псевдоним, количество убийств и смертей, игроки сортируются по количеству убийств: у кого выше - тот сверху. Play Zone - блок Canvas, отрисовывающий векторную графику. Chat - блок с общим чатом, сообщения сортируются по следующему принципу: новые сообщения внизу, старые сверху.

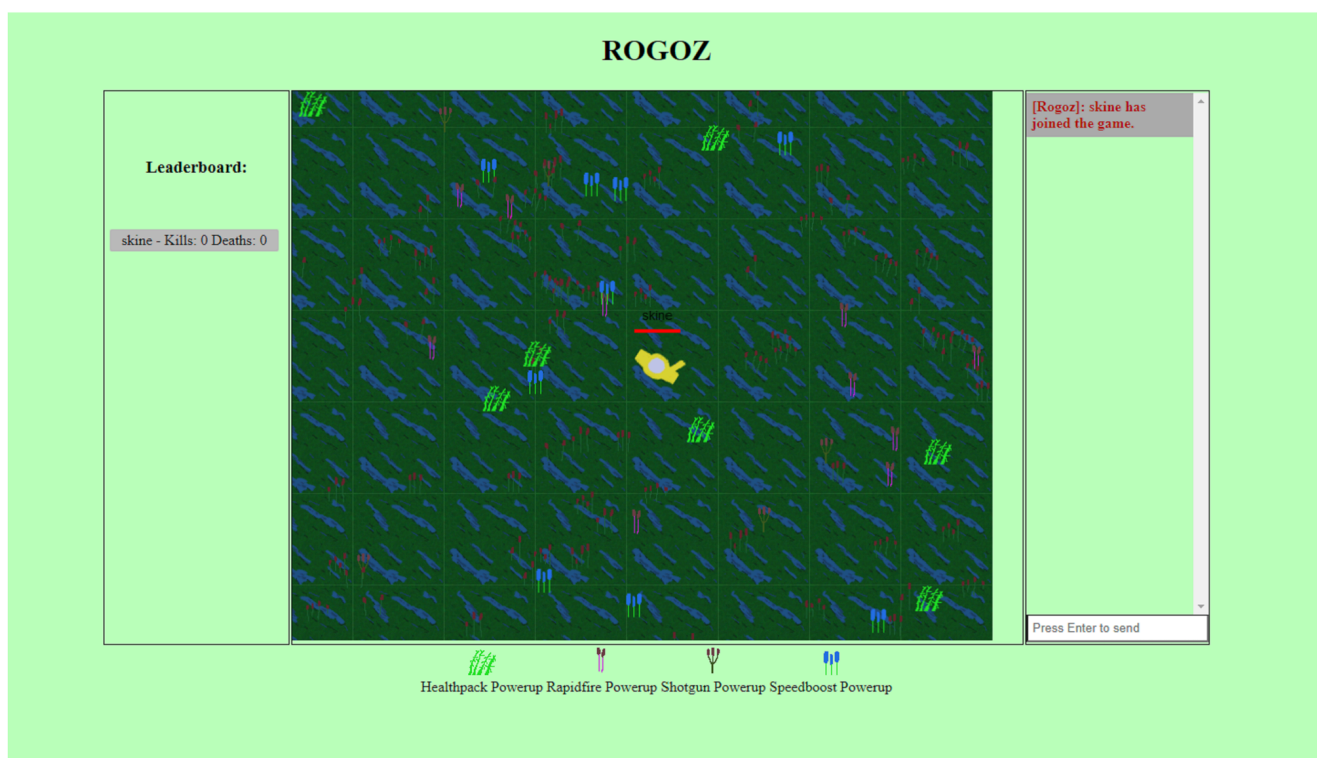


Рисунок 3.4 – Вход первого игрока

Для этого использовался элемент HTML контейнер, который был поделен на 3 части - таблица лидеров, основной элемент игры canvas и общий чат:

“Смотри приложение №8”

Для простоты использования, под общими элементами были размещены подсказки - рисунки улучшателей и их свойства:

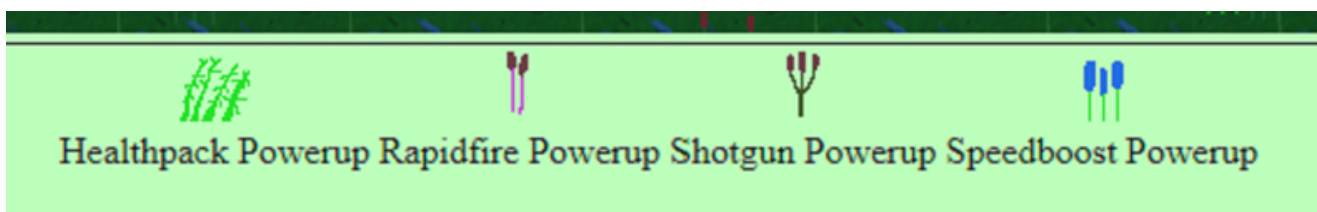


Рисунок 3.5 – Улучшатели

Присоединение каждого последующего игрока оглашается в чате, так же список игроков можно увидеть в таблице лидеров:

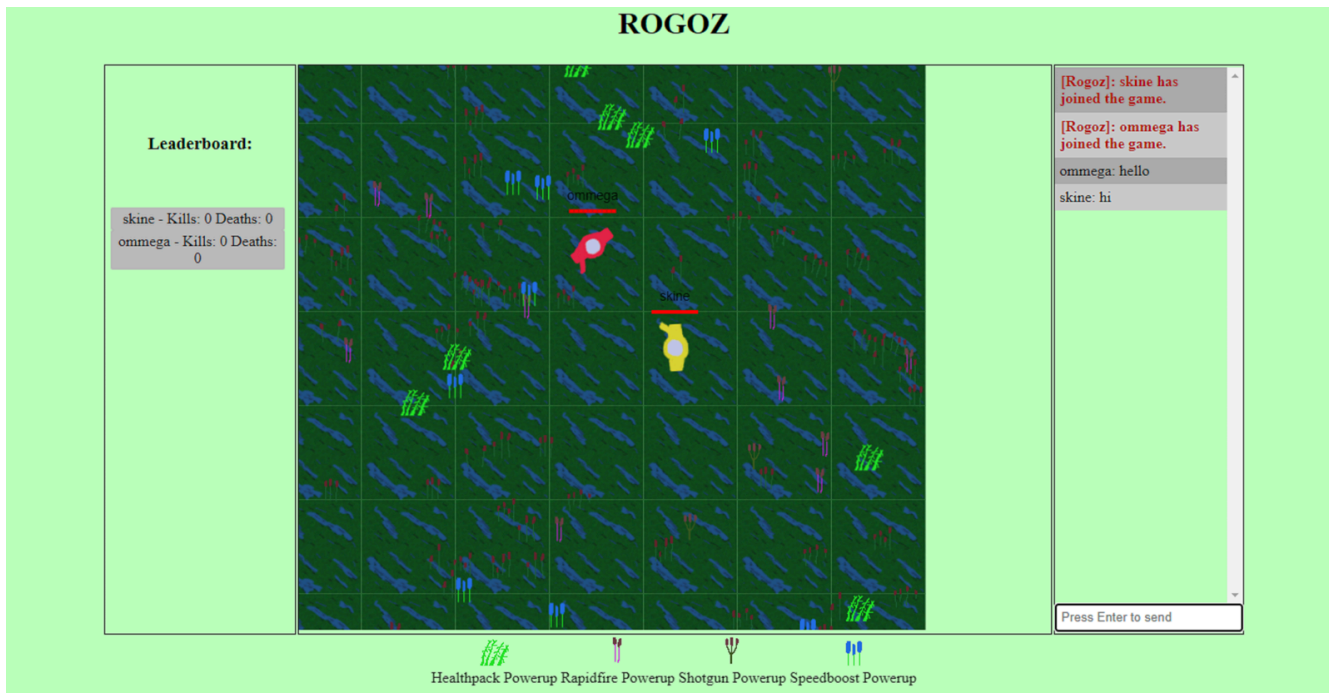


Рисунок 3.6 – Второй игрок присоединился



Рисунок 3.7 – Общий чат

4. Проектно-технологическая часть

4.1 Тестирование и отладка готового модуля.

В таблице 4.1 приведены результаты тестирования готового модуля методом чёрного ящика.

Таблица 4.1 – Тест кейсы

Действие	Ожидаемый результат	Результат теста
Подключение к серверу	Выводится окно, предлагающее ввести никнейм	Выполнен
Ввод никнейма длиной более 20 символов	Выводится сообщение о слишком большой длине никнейма или пустом никнейме	Выполнен
Попыка пройти дальше не вводя никнейм	Выводится сообщение о слишком большой длине никнейма или пустом никнейме	Выполнен
Ввод правильного никнейма	Загружается чат, таблица лидеров и основной элемент с игровым пространством	Выполнен
Ввод сообщения в чате	Сообщение отображается в чате от вашего имени и видно всем пользователям на сервере	Выполнен
Нажатие кнопок wasd	Производится движение персонажа на поле	Выполнен

Таблица 4.1 – Продолжение

Действие	Ожидаемый результат	Результат теста
Нажатие левой кнопки мыши	производится бросок рогоза	Выполнен
Нокаутирование другого пользователя	персонаж игрока пропадает и появляется в другом месте, таблица лидеров изменяется, засчитываются очки	Выполнен
Подбирание улучшителя	Характеристики игрока улучшаются на некоторое время в зависимости от выбранного улучшения	Выполнен
Нокаутирование собственного персонажа	Персонаж появляется в новом месте с полной шкалой здоровья, таблица лидеров изменяется, засчитываются очки	Выполнен

4.2 Руководство администратора.

Для установки системы на персональный компьютер/сервер необходимо следующее окружение:

1. Операционная система Windows 7 и выше.
2. Наличие доступа к интернету.
3. Свободное место на диске не менее 150 Мб.
4. Установленный Node.js версии v20.13.1 и выше.

Порядок развёртывания:

1. Скачать файл zip, распаковать в любой удобной папке.
2. С помощью командной строки перейти в директорию этой папки, используя команду cd

```
C:\Users\krisk>cd c:\Rogoz  
  
c:\Rogoz>
```

Рисунок 4.1 - Перемещение в директорию игры

3. Установить последний пакет зависимостей npm используя команду npm install

```
C:\Users\krisk>cd c:\Rogoz-main  
  
c:\Rogoz-main>npm install  
npm WARN deprecated @types/socket.io@3.0.2: This is a stub types definition. socket.io provides its own type definitions  
, so you do not need this installed.  
npm WARN deprecated @types/del@4.0.0: This is a stub types definition. del provides its own type definitions, so you do  
not need this installed.  
npm WARN deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache i  
f you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerfu  
l.  
npm WARN deprecated source-map-url@0.4.1: See https://github.com/lydell/source-map-url#deprecated  
npm WARN deprecated flatten@1.0.3: flatten is deprecated in favor of utility frameworks such as lodash.  
npm WARN deprecated urix@0.1.0: Please see https://github.com/lydell/urix#deprecated  
npm WARN deprecated browserslist@1.7.7: Browserslist 2 could fail on reading Browserslist >3.0 config used in other tool  
s.  
npm WARN deprecated source-map-resolve@0.5.3: See https://github.com/lydell/source-map-resolve#deprecated  
npm WARN deprecated chokidar@2.1.8: Chokidar 2 does not receive security updates since 2019. Upgrade to chokidar 3 with  
15x fewer dependencies  
npm WARN deprecated resolve-url@0.2.1: https://github.com/lydell/resolve-url#deprecated  
npm WARN deprecated svgo@0.7.2: This SVG version is no longer supported. Upgrade to v2.x.x.  
  
added 893 packages, and audited 894 packages in 1m  
  
83 packages are looking for funding  
  run `npm fund` for details  
  
42 vulnerabilities (33 moderate, 9 high)  
  
To address issues that do not require attention, run:  
  npm audit fix  
  
To address all issues possible (including breaking changes), run:  
  npm audit fix --force  
  
Some issues need review, and may require choosing  
a different dependency.  
  
Run `npm audit` for details.  
  
c:\Rogoz-main>
```

Рисунок 4.2 - Установка npm в директорию приложения

Возможны некоторые предупреждения, но это не повлияет на работоспособность программы.

4. Установить gulp командой npm install --global gulp-cli

```
c:\Rogoz>npm install --global gulp-cli
```

Рисунок 4.3 - Установка сборщика gulp

5. Собрать приложение используя команду `gulp compile`

```
c:\Rogoz-main>gulp compile
[22:55:02] Using gulpfile c:\Rogoz-main\gulpfile.js
[22:55:02] Starting 'compile'...
[22:55:02] Starting 'copyHTML'...
[22:55:02] Starting 'copyIMG'...
[22:55:02] Starting 'compileTypescriptServer'...
[22:55:02] Starting 'compileTypescriptClient'...
[22:55:02] Starting 'compileLess'...
[22:55:04] Finished 'copyHTML' after 1.73 s
[22:55:04] Finished 'compileLess' after 1.73 s
[22:55:05] Finished 'compileTypescriptServer' after 3.06 s
[22:55:05] Finished 'copyIMG' after 3.06 s
[22:55:05] Finished 'compileTypescriptClient' after 3.17 s
[22:55:05] Finished 'compile' after 3.17 s

c:\Rogoz-main>
```

Рисунок 4.4 - Сборка приложения

6. Для дальнейшего запуска сервера необходимо использовать команду `npm run start`

```
c:\Rogoz>npm run start

> rogoz@1.0.0 start
> node --experimental-specifier-resolution=node dist/server.js

Starting server on port 5000
|
```

Рисунок 4.5 - Запуск сервера

7. Для остановки сервера необходимо прервать процесс командной строки, либо использовать Сочетание клавиш `Ctrl+C`.
8. Для изменения порта запуска сервера необходимо открыть файл в директории игры: `C:\Rogoz\dist\server.js` и изменить константу `const PORT = process.env.PORT`

4.3 Руководство пользователя.

Для присоединения к игровому процессу компьютер пользователя должен соответствовать следующим требованиям:

1. Иметь браузер на движке chromium и поддержкой HTML5.
2. Иметь надёжное подключение к интернету не менее 10 мбит/сек.
3. Иметь средства управления - клавиатура + мышь\трекпад.
4. Иметь более 2 Мб оперативной памяти.

Для подключения к серверу, игрок должен ввести адрес сервера в поисковой строке браузера. Далее игрока встретит приветственное окно: в поле nickname нужно ввести свой псевдоним (поле не должно быть пустое и псевдоним не должен быть более 20 символов в длину). После ввода необходимо нажать на кнопку Play:



Рисунок 4.6 - Приветственное окно

После нажатия - игрока перекинет на главное окно. Двигаться в поле можно используя клавиши wasd. Кидать рогоз можно зажав левую кнопку мыши. Слева от игрового поля изображена таблица лидеров, справа - общий чат. Для написания сообщения необходимо левой кнопкой мыши кликнуть на поле “Press Enter to send”, далее ввести текст и нажать клавишу Enter на клавиатуре для отправки сообщения. Снизу, под игровым полем можно увидеть 4 изображения

улучшателей с описанием, данная подсказка призвана помочь новичкам адаптироваться к игровому процессу.

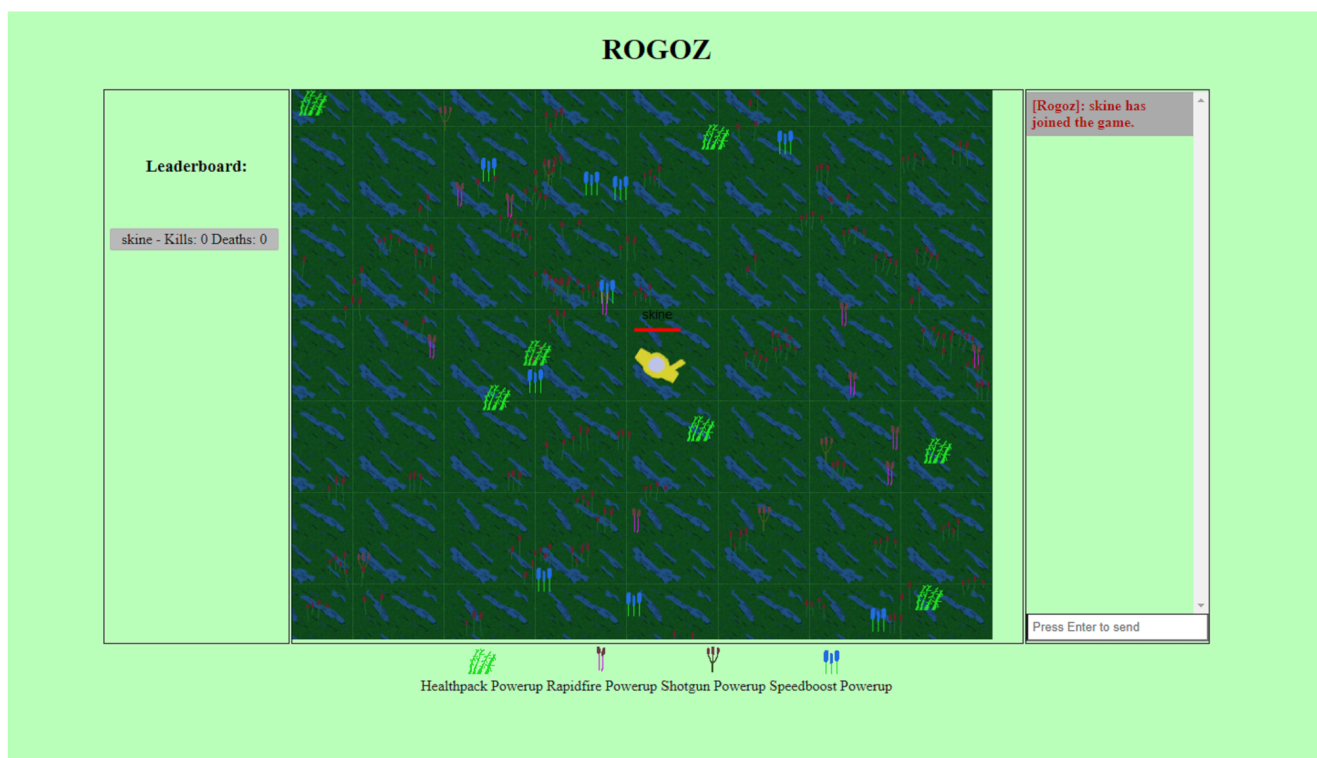


Рисунок 4.7 – Главное окно

Для отключения от сервера достаточно просто свернуть окно в браузере.

Заключение

В ходе выполнения данной курсовой работы были решены задачи и была достигнута цель проекта. Было разработано приложение, реализующее функции:

- Отправка данных клиентом серверу
- Обработка данных на сервере
- Приём данных клиентом
- Отрисовка данных на клиенте

В будущем проект может получать обновления, которые будут включать появление новых функций, исправления визуальных недочетов и ускорения работы некоторых алгоритмов.

Список литературы

1. Башлы, П., Н. Современные сетевые технологии / П. Н. Башлы. - М.: ГЛТ, 2006. - 334 с.
2. Будылдина, Н.В. Сетевые технологии высокоскоростной передачи данных: Учебное пособие для вузов / Н.В. Будылдина, В.П. Шувалов. - М.: РиС, 2016. - 342 с.
3. Галушкин, А.И. Нейросетевые технологии в России (1982–2010) / А.И. Галушкин. - М.: ГЛТ, 2012. - 316 с.
4. Елисеева, И.И. Основы информационных и телекоммуникационных технологий. Часть 3. Сетевые информационные технологии. Учебное пособие / И.И. Елисеева. - М.: Финансы и статистика, 2005. - 224 с.
5. Иртегов, Д. Введение в сетевые технологии. / Д. Иртегов. - СПб.: ВHV, 2004. - 560 с.
6. Кузьменко, Н.Г. Компьютерные сети и сетевые технологии / Н.Г. Кузьменко. - СПб.: Наука и техника, 2013. - 368 с.
7. Сидоров В.Н., Сломинская Е.Н., Полникова Т.В., Макарова О.Ю. Оформление графической части выпускной квалификационной работы. Учебное пособие. М.: МГТУ им. Н.Э. Баумана, 2016.

Приложение №1

```
io.on('connection', (socket) => {
  socket.on(Constants.SOCKET.NEW_PLAYER, (name, callback) => {
    game.addNewPlayer(name, socket);
    io.emit(Constants.SOCKET.CHAT_SERVER_CLIENT, {
      name: CHAT_TAG,
      message: `${name} has joined the game.`,
      isNotification: true,
    });
    callback();
  });
  socket.on(Constants.SOCKET.PLAYER_ACTION, (data) => {
    game.updatePlayerOnInput(socket.id, data);
  });
  socket.on(Constants.SOCKET.CHAT_CLIENT_SERVER, (message) => {
    io.emit(Constants.SOCKET.CHAT_SERVER_CLIENT, {
      name: game.getPlayerNameBySocketId(socket.id),
      message: message,
      isNotification: false,
    });
  });
  socket.on(Constants.SOCKET.DISCONNECT, () => {
    const name = game.removePlayer(socket.id);
    io.sockets.emit(Constants.SOCKET.CHAT_SERVER_CLIENT, {
      name: CHAT_TAG,
      message: `${name} has left the game.`,
      isNotification: true,
    });
  });
});
```

Приложение №2

```
io.on('connection', (socket) => {
  socket.on(Constants.SOCKET.NEW_PLAYER, (name, callback) => {
    game.addNewPlayer(name, socket);
    io.emit(Constants.SOCKET.CHAT_SERVER_CLIENT, {
      name: CHAT_TAG,
      message: `${name} has joined the game.`,
      isNotification: true,
    });
    callback();
  });
  socket.on(Constants.SOCKET.PLAYER_ACTION, (data) => {
    game.updatePlayerOnInput(socket.id, data);
  });
  socket.on(Constants.SOCKET.CHAT_CLIENT_SERVER, (message) => {
    io.emit(Constants.SOCKET.CHAT_SERVER_CLIENT, {
      name: game.getPlayerNameBySocketId(socket.id),
      message: message,
      isNotification: false,
    });
  });
  socket.on(Constants.SOCKET.DISCONNECT, () => {
    const name = game.removePlayer(socket.id);
```

```

        io.sockets.emit(Constants.SOCKET.CHAT_SERVER_CLIENT, {
            name: CHAT_TAG,
            message: `${name} has left the game.`,
            isNotification: true,
        });
    });
});

```

Приложение №3

```

sendState() {
    const players = [...this.players.values()];
    this.clients.forEach((_client, socketID) => {
        const currentPlayer = this.players.get(socketID);
        this.clients.get(socketID).emit(Constants.SOCKET.UPDATE, {
            self: currentPlayer,
            players: players,
            projectiles: this.projectiles,
            powerups: this.powerups,
        });
    });
}

```

Приложение №4

```

update() {
    const currentTime = Date.now();
    this.deltaTime = currentTime - this.lastUpdateTime;
    this.lastUpdateTime = currentTime;
    const entities = [
        ...this.players.values(),
        ...this.projectiles,
        ...this.powerups,
    ];
    entities.forEach((entity) => {
        entity.update(this.lastUpdateTime, this.deltaTime);
    });
    for (let i = 0; i < entities.length; ++i) {
        for (let j = i + 1; j < entities.length; ++j) {
            let e1 = entities[i];
            let e2 = entities[j];
            if (!e1.collided(e2)) {
                continue;
            }
            if (e1 instanceof Bullet && e2 instanceof Player) {
                e1 = entities[j];
                e2 = entities[i];
            }
            if (e1 instanceof Player && e2 instanceof Bullet && e2.source !== e1) {
                e1.damage(e2.damage);
                if (e1.isDead()) {
                    e1.spawn();
                    e1.deaths++;
                    e2.source.kills++;
                }
                e2.destroyed = true;
            }
            if (e1 instanceof Powerup && e2 instanceof Player) {
                e1 = entities[j];
                e2 = entities[i];
            }
        }
    }
}

```

```

        if (e1 instanceof Player && e2 instanceof Powerup) {
            e1.applyPowerup(e2);
            e2.destroyed = true;
        }
        if (e1 instanceof Bullet &&
            e2 instanceof Bullet &&
            e1.source !== e2.source) {
            e1.destroyed = true;
            e2.destroyed = true;
        }
        if ((e1 instanceof Powerup && e2 instanceof Bullet) ||
            (e1 instanceof Bullet && e2 instanceof Powerup)) {
            e1.destroyed = true;
            e2.destroyed = true;
        }
    }
}
this.projectiles = this.projectiles.filter((projectile) => !projectile.destroyed);
this.powerups = this.powerups.filter((powerup) => !powerup.destroyed);
while (this.powerups.length < Constants.POWERUP_MAX_COUNT) {
    this.powerups.push(Powerup.create());
}
}

```

Приложение №5

```

updateOnInput(data) {
    if ((data.up && data.down) || (!data.up && !data.down)) {
        this.velocity = Vector.zero();
    }
    else if (data.up) {
        this.velocity = Vector.fromPolar(this.speed, this.manAngle);
    }
    else if (data.down) {
        this.velocity = Vector.fromPolar(-this.speed, this.manAngle);
    }
    if ((data.left && data.right) || (!data.left && !data.right)) {
        this.turnRate = 0;
    }
    else if (data.right) {
        this.turnRate = Constants.PLAYER_TURN_RATE;
    }
    else if (data.left) {
        this.turnRate = -Constants.PLAYER_TURN_RATE;
    }
    this.handAngle = data.handAngle;
}

```

Приложение №6

```

static createFromPlayer(player, angleDeviation) {
    const angle = player.handAngle + angleDeviation;
    return new Bullet(player.position.copy(),
        Vector.fromPolar(Constants.BULLET_SPEED, angle), angle, player);
}

```

```

update(_lastUpdateTime, deltaTime) {
    const distanceStep = Vector.scale(this.velocity, deltaTime);
    this.position.add(distanceStep);
    if (!this.inWorld()) {
        this.destroyed = true;
        return;
    }
    this.distanceTraveled += distanceStep.mag2;
    if (this.distanceTraveled > Math.pow(Constants.BULLET_MAX_TRAVEL_DISTANCE, 2)) {
        this.destroyed = true;
    }
}

```

Приложение №7

```

draw() {
    if (this.self) {
        this.drawing.clear();
        this.drawing.drawTiles();
        this.projectiles.forEach(this.drawing.drawBullet.bind(this.drawing));
        this.powerups.forEach(this.drawing.drawPowerup.bind(this.drawing));
        this.drawing.drawMan(true, this.self);
        this.players
            .filter((player) => { var _a; return player.socketID !== ((_a =
this.self) === null || _a === void 0 ? void 0 : _a.socketID); })
            .forEach((man) => this.drawing.drawMan(false, man));
    }
}

```

Приложение №8

```

<div id='container'>
    <table>
        <tr>
            <th colspan='3'>
                <h1>ROGOZ</h1>
            </th>
        </tr>
        <tr>
            <td id='leaderboard-container'>
                <h3>Leaderboard:</h3>
                <ul id='leaderboard'></ul>
            </td>
            <td id='canvas-container'>
                <canvas id='canvas'></canvas>
            </td>
            <td id='chat-container'>
                <ul id='chat-display'>
                </ul>
                <input id='chat-input' type='text'
                    placeholder='Press Enter to send' name='chat' />
            </td>
        </tr>
    </table>

```

```

        </td>
    </tr>
    <tr>
        <td id='icons-container' colspan='3'>
            <div>
                <img src='/img/health_pack_powerup.png' />
                <div>Healthpack Powerup</div>
            </div>
            <div>
                <img src='/img/rapidfire_powerup.png' />
                <div>Rapidfire Powerup</div>
            </div>
            <div>
                <img src='/img/shotgun_powerup.png' />
                <div>Shotgun Powerup</div>
            </div>
            <div>
                <img src='/img/speedboost_powerup.png' />
                <div>Speedboost Powerup</div>
            </div>
        </td>
    </tr>
</table>
</div>

```